

Travaux Pratiques 4

Blockchain Peer-to-Peer avec Flask

Décentralisation, Propagation et Consensus Distribué

Abdoullah Ichou
EPSI — Auxerre

19 mai 2026

Durée : 1h30 **Groupes :** Binôme (chaque poste = un nœud du réseau)

Prérequis : Python 3.8+, Flask, Requests

Installation : `pip install flask requests`

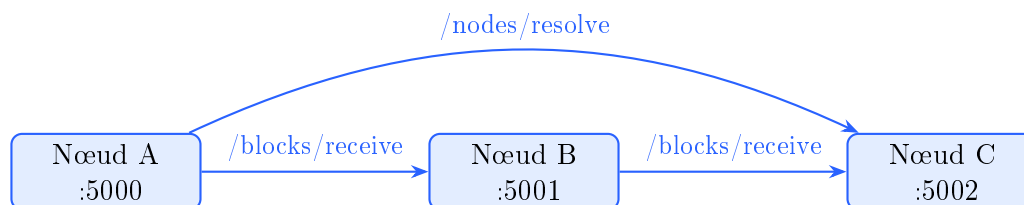
Objectifs pédagogiques :

- Implémenter un nœud blockchain complet exposé via une API REST
- Enregistrer des nœuds pairs (*peers*) et propager des blocs
- Implémenter la règle de consensus de la chaîne la plus longue
- Observer et analyser un *fork* en conditions réelles

1 Architecture du système

Chaque nœud Flask expose 6 routes REST :

Méthode	Route	Rôle
GET	/chain	Retourne la blockchain complète (JSON)
POST	/transactions/new	Ajoute une transaction au mempool
GET	/mine	Mine un nouveau bloc avec les transactions en attente
POST	/nodes/register	Enregistre de nouveaux nœuds pairs
GET	/nodes/resolve	Exécute l'algorithme de consensus
POST	/blocks/receive	Reçoit un bloc propagé par un pair



Important — Ports : Chaque binôme lance son nœud sur un port différent. Utilisez les ports 5000, 5001, 5002, etc. selon les instructions du professeur. Si tout le monde est sur la même machine, le réseau est local ; en réseau LAN, utilisez l'IP réelle de chaque poste.

2 Code de démarrage

Fichier : tp4_node_starter.py

```

1  #!/usr/bin/env python3
2  """
3  TP 4 --- Blockchain P2P avec Flask : Code de démarrage
4  Complétez les parties # TODO dans l'ordre numérique.
5
6  Lancement : python tp4_node_starter.py --port 5000
7  """
8
9  import hashlib
10 import time
11 import json
12 import argparse
13 import requests
14 from flask import Flask, jsonify, request
15
16 app = Flask(__name__)
17
18 # =====
19 # MODÉLE DE DONNÉES
20 # =====
21
22 blockchain = []      # Liste de blocs (dicts)
23 mempool    = []      # Transactions en attente de minage
24 peers      = set()    # Ensemble d'URL des nœuds pairs
25
26 DIFFICULTY = 3       # Nombre de zéros requis (ajustable)
27
28
29 def calculate_hash(index, previous_hash, timestamp, data, nonce):
30     """Calcule le SHA-256 d'un bloc à partir de ses champs."""
31     content = f"{index}{previous_hash}{timestamp}{json.dumps(data,
32         sort_keys=True)}{nonce}"
33     return hashlib.sha256(content.encode()).hexdigest()
34
35 def create_block(data, previous_hash):
36     """
37     Crée et mine un nouveau bloc.
38
39     Args:
40         data          -- liste de transactions
41         previous_hash -- hash du bloc précédent
42
43     Returns:
44         Un dict représentant le bloc miné.
45     """
46     # TODO 1 : Implémenter la création et le minage d'un bloc
47     # Structure du bloc :
48     #   index          = len(blockchain)

```

```

49     # timestamp      = time.time()
50     # data            = data (liste de transactions)
51     # previous_hash  = previous_hash
52     # nonce           = 0 (incr\`ement\`e jusqu\`a ce que le hash soit
    valide)
53     # hash            = calculate_hash(...)
54     # Boucle : incr\`ementer nonce jusqu\`a ce que hash.startswith("0"
    * DIFFICULTY)
55     # Retourner le dict du bloc
56     pass
57
58
59 def create_genesis_block():
60     """Cr\`ee le bloc de gen\`ese (index=0, previous_hash='0'*64)."""
61     return create_block(["Genesis Block"], "0" * 64)
62
63
64 # =====
65 # ROUTES DE L'API REST
66 # =====
67
68 @app.route('/chain', methods=['GET'])
69 def get_chain():
70     """Retourne la blockchain compl\`ete."""
71     return jsonify({
72         'chain': blockchain,
73         'length': len(blockchain)
74     }), 200
75
76
77 @app.route('/transactions/new', methods=['POST'])
78 def new_transaction():
79     """Ajoute une transaction au mempool."""
80     body = request.get_json()
81
82     # V\`erification des champs requis
83     required = ['sender', 'recipient', 'amount']
84     if not all(k in body for k in required):
85         return jsonify({'error': 'Champs manquants : sender, recipient,
    amount'}), 400
86
87     # TODO 2 : Ajouter la transaction \`a mempool et retourner un
    message de confirmation
88     # Format d'une transaction : {'sender': ..., 'recipient': ..., '
    amount': ...}
89     pass
90
91
92 @app.route('/mine', methods=['GET'])
93 def mine():
94     """Mine un nouveau bloc avec les transactions du mempool."""
95     # TODO 3 :
96     # 1. V\`erifier que le mempool n'est pas vide (sinon retourner une
    erreur 400)
97     # 2. R\`ecup\`erer le hash du dernier bloc (blockchain[-1]['hash'])
98     # 3. Cr\`eer un nouveau bloc avec create_block(mempool.copy(),
    last_hash)
99     # 4. Vider le mempool

```

```

100     # 5. Ajouter le bloc \à blockchain
101     # 6. Propager le bloc aux pairs avec broadcast_block(new_block)
102     # 7. Retourner les infos du bloc min\é (200)
103     pass
104
105
106 @app.route('/nodes/register', methods=['POST'])
107 def register_nodes():
108     """Enregistre de nouveaux nœuds pairs."""
109     body = request.get_json()
110     nodes_list = body.get('nodes', [])
111
112     if not nodes_list:
113         return jsonify({'error': 'Fournir une liste de nœuds'}), 400
114
115     # TODO 4 : Ajouter chaque URL de nœud dans le set peers
116     #           Retourner la liste des pairs enregistrés
117     pass
118
119
120 @app.route('/nodes/resolve', methods=['GET'])
121 def resolve_conflicts():
122     """
123     Algorithme de consensus : remplace la chaîne locale
124     par la chaîne la plus longue trouvée parmi les pairs.
125     """
126     # TODO 5 : Implémenter consensus()
127     replaced = consensus()
128     if replaced:
129         return jsonify({'message': 'Chaîne remplacée', 'new_chain':
130                         blockchain}), 200
131     else:
132         return jsonify({'message': 'Chaîne déjà jour', 'chain':
133                         blockchain}), 200
134
135
136 @app.route('/blocks/receive', methods=['POST'])
137 def receive_block():
138     """Reçoit un bloc propagé par un pair et le valide."""
139     block = request.get_json()
140
141     # TODO 6 : Valider le bloc reçu
142     # Vérifier :
143     # - block['index'] == len(blockchain) (bloc attendu)
144     # - block['previous_hash'] == blockchain[-1]['hash'] (chaîne
145     #   correcte)
146     # - block['hash'].startswith("0" * DIFFICULTY) (PoW valide)
147     # - block['hash'] == calculate_hash(...) (intégrité)
148     # Si valide : ajouter à blockchain, retourner 200
149     # Si index trop grand : appeler consensus() pour se synchroniser
150     # Sinon : rejeter (409)
151     pass
152
153 # =====
154 # FONCTIONS UTILITAIRES
155 # =====

```

```

155 def broadcast_block(block):
156     """
157     Propage un bloc nouvellement min\`e \`a tous les pairs.
158     """
159     for peer in peers:
160         try:
161             requests.post(f"{peer}/blocks/receive", json=block, timeout
162                           =3)
163         except requests.exceptions.ConnectionError:
164             pass # Le pair est peut-\`etre hors ligne
165
166 def consensus():
167     """
168     R\`egle de la cha\`ne la plus longue.
169     Parcourt tous les pairs, r\`ecup\`ere leur cha\`ne, adopte
170     la plus longue si elle est valide et plus longue que la n\`otre.
171
172     Returns:
173         True si la cha\`ne locale a \`et\`e remplac\`ee, False sinon
174     """
175     global blockchain
176
177     # TODO 7 :
178     # Pour chaque pair dans peers :
179     # - GET {peer}/chain
180     # - Comparer la longueur avec len(blockchain)
181     # - Valider la cha\`ne distante avec is_valid_chain(chain)
182     # - Conserver la plus longue valide
183     # Si une cha\`ne plus longue valide est trouv\`ee, la substituer
184     # Retourner True si remplacement, False sinon
185     pass
186
187
188 def is_valid_chain(chain):
189     """
190     V\`erifie l'int\`egrit\`e d'une blockchain compl\`ete.
191
192     V\`erifie pour chaque bloc (i >= 1) :
193     1. Cha\`nage (previous_hash du bloc i == hash du bloc i-1)
194     2. Int\`egrit\`e (hash enregistr\`e == hash recalcul\`e)
195     3. PoW (hash commence par DIFFICULTY z\`eros)
196     """
197     # TODO 8 : Impl\`ementer la validation compl\`ete
198     pass
199
200
201 # =====
202 # POINT D'ENTR\`EE
203 # =====
204
205 if __name__ == '__main__':
206     parser = argparse.ArgumentParser(description='N\`oeud Blockchain
207                                         P2P')
208     parser.add_argument('--port', type=int, default=5000, help='Port du
209                                         n\`oeud (d\`efaut: 5000)')
210     args = parser.parse_args()

```

```

209
210 # Initialiser avec le bloc de gen\`ese
211 blockchain.append(create_genesis_block())
212 print(f"[INIT] Bloc de gen\`ese cr\`e\`e : {blockchain[0]['hash'][:20]}...")
213 print(f"[INFO] N\`oeud d\`emarr\`e sur http://localhost:{args.port}")
214 print(f"[INFO] Difficult\`e PoW : {DIFFICULTY} z\`ero(s)")
215
216 app.run(host='0.0.0.0', port=args.port)

```

3 Exercices

Exercice 1 — Compléter le code [TODO 1 à 8]

Complétez toutes les parties # TODO du squelette. Testez votre nœud seul en premier :

1. Démarrez votre nœud : `python tp4_node_starter.py -port 5000`
2. Ajoutez une transaction (depuis un autre terminal) :

```

curl -X POST http://localhost:5000/transactions/new \
-H "Content-Type: application/json" \
-d '{"sender": "Alice", "recipient": "Bob", "amount": 10}'

```

3. Minez un bloc : `curl http://localhost:5000/mine`
4. Consultez la chaîne : `curl http://localhost:5000/chain`

Exercice 2 — Réseau P2P en binôme

Deux nœuds (Nœud A sur :5000, Nœud B sur :5001) :

1. Démarrez chacun votre nœud sur votre poste.
2. Enregistrez-vous mutuellement :

```

# Sur Nœud A : enregistrer Nœud B
curl -X POST http://localhost:5000/nodes/register \
-H "Content-Type: application/json" \
-d '{"nodes": ["http://localhost:5001"]}'

# Sur Nœud B : enregistrer Nœud A
curl -X POST http://localhost:5001/nodes/register \
-H "Content-Type: application/json" \
-d '{"nodes": ["http://localhost:5000"]}'

```

3. Nœud A mine 3 blocs. Nœud B en mine 1.
4. Exécutez le consensus sur Nœud B : `curl http://localhost:5001/nodes/resolve`
5. **Question** : Quelle chaîne Nœud B adopte-t-il ? Pourquoi ?

Exercice 3 — Simulation d'un fork

1. Stoppez la propagation (commentez temporairement `broadcast_block` dans `mine`).

2. Nœud A et Nœud B minent simultanément un bloc (même index).
3. Rétablissez la propagation.
4. Appelez `/nodes/resolve` sur les deux nœuds.
5. **Que se passe-t-il ?** Lequel garde son bloc ? Pourquoi ?

Exercice 4 (Bonus) — Attaque des 51%

1. Un réseau de 3 nœuds (A, B, C) mine chacun 2 blocs sur la chaîne normale.
2. Le nœud C crée une *bifurcation secrète* depuis le bloc 1 et mine 5 blocs sans les propager.
3. C propage alors sa chaîne via `/nodes/resolve`.
4. **Que se passe-t-il ?** La chaîne de C est-elle adoptée ? Qu'est-ce que cela illustre ?